

Phase 2 Assignment 2B - Backend API Projects

Goal: Master Phase 1 and Phase 2 by building realistic, modular FastAPI backends that combine multiple concepts in the same application. Scope: Only use topics taught in Lessons 1-19. Standard: Harder than the Phase 1 project assignment. These projects should feel like mini capstones.

Allowed Scope

Use only concepts from Phase 1 and Phase 2:

- FastAPI app setup and route decorators
- Path parameters, query parameters, and request bodies
- Pydantic models, nested models, `Field()`, `field_validator`, `model_validator`, and `model_config`
- Response basics and `response_model`
- Separate input/output/update models
- `status_code`, `HTTPException`, custom exception handlers, and validation error handling
- `Depends()`, sub-dependencies, class-based dependencies, and `yield` dependencies
- Middleware
- `APIRouter`
- Form data and file uploads
- Headers and cookies
- JSON responses and standard FastAPI response objects
- Static JSON file persistence and local file storage
- Python standard library only

Do not use:

- Databases, ORMs, SQLAlchemy, Alembic, SQLModel, MongoDB, Redis
- JWT, OAuth2, password hashing, login systems, or real authentication
- Background tasks, WebSockets, SSE, streaming, rate limiting libraries, caching libraries
- Testing frameworks as a required part of the assignment
- Docker, deployment, CI/CD, environment-management packages
- Any Phase 3+ feature, even if it would be useful in a real system

Important: You may use headers, cookies, and request-scoped dependency checks, but do not turn them into a real authentication or authorization system. If a project needs role-like behavior, treat it as a simple validated request input, not production security.

Global Rules

- Build backend APIs only.

- Do not build a frontend.
- Do not provide solutions, code, pseudocode, or implementation hints.
- Use routers and split code into multiple files.
- Use `response_model` and separate input/output models wherever appropriate.
- Use global or router-level exception handling where it improves consistency.
- Use middleware where it is a better fit than repeating logic in endpoints.
- Use dependencies for reusable request parsing, validation, and contextual logic.
- Use JSON files for data persistence and local folders for uploaded files.
- Each project must be runnable with `uvicorn main:app --reload`.
- Every project should be designed as if another developer will maintain it after you.

Recommended persistence pattern:

- `data/*.json` for structured records
- `uploads/` for uploaded files
- IDs generated from existing records
- Response models used to hide internal file paths, internal notes, or other non-public fields

Recommended evaluation mindset:

- Clean API design
- Correct validation
- Consistent error handling
- Clear separation between request models and response models
- Appropriate use of routers, dependencies, and middleware

Project 1 - Vendor Onboarding And Compliance Review API

Difficulty Level

Hard

Estimated Completion Time

10-14 hours

Project Overview

Build an API for a company that onboards third-party vendors and collects compliance documents before they are approved for partnership.

This project should combine file uploads, routers, dependencies, response models, exception handling, middleware, headers, and workflow rules in one realistic application.

Problem Statement

The operations team needs a backend system to:

- Register vendors
- Track onboarding status
- Upload compliance documents
- Review submitted documents
- Approve or reject vendor onboarding
- Filter vendors by status, risk level, and missing requirements

The system should behave like an internal operations API, not a toy CRUD app.

Functional Requirements

- Create vendor profiles
- List and search vendors
- View vendor details
- Update vendor metadata
- Upload compliance documents for a vendor
- List documents for a vendor
- Review a document as approved or rejected
- Record document review notes
- Mark a vendor as approved only when all required documents are approved
- Return onboarding summaries and pending-review counts

Non-Functional Requirements

- Use modular routing
- Persist vendor and document metadata in JSON files
- Save uploaded files in a local folder
- Return consistent error response shapes
- Add middleware for request timing and request ID tracing

API Requirements

- Use routers such as `vendors`, `documents`, and `reviews`
- Use input/output/update models instead of one model for everything
- Use a response model that hides internal file-system paths
- Use a dependency to load and validate vendor existence

- Use a dependency to parse pagination and filter parameters
- Read reviewer identity from a required header such as `X-Reviewer-Name`
- Use form fields plus file upload for document submission

Required API Endpoints

Method	Path	Purpose
POST	<code>/vendors</code>	Create vendor
GET	<code>/vendors</code>	List/search/filter vendors
GET	<code>/vendors/{vendor_id}</code>	Get one vendor
PUT	<code>/vendors/{vendor_id}</code>	Replace vendor details
PATCH	<code>/vendors/{vendor_id}/status</code>	Update onboarding status through controlled input
GET	<code>/vendors/{vendor_id}/summary</code>	Get onboarding summary
POST	<code>/vendors/{vendor_id}/documents</code>	Upload compliance document
GET	<code>/vendors/{vendor_id}/documents</code>	List vendor documents
GET	<code>/documents/{document_id}</code>	Get one document
POST	<code>/documents/{document_id}/review</code>	Approve or reject document
GET	<code>/dashboard/compliance</code>	Return aggregate onboarding metrics

Request And Response Expectations

Vendor creation request should include:

- `name`
- `business_type`
- `contact_email`
- `contact_phone`
- `country`
- `risk_level`
- `required_documents`

Vendor creation response should:

- Return `201`
- Include generated `id`
- Include onboarding status
- Exclude internal reviewer notes

Document upload request should use multipart form data and include:

- `document_type`
- `expires_on`
- `notes`
- `file`

Document review request should include:

- `decision`
- `review_note`

List endpoints should support:

- search
- status filter
- risk filter
- missing document filter
- pagination
- sorting by vendor name or created date

Validation Requirements

- Vendor name: 2-120 characters
- `business_type`: one of a controlled set
- `contact_email`: valid email format
- `contact_phone`: validated format
- `country`: 2-80 characters
- `risk_level`: one of `low`, `medium`, `high`
- `required_documents`: non-empty list
- `document_type`: must be one of the required vendor document types
- `decision`: one of `approved`, `rejected`
- `review_note`: 5-1000 characters
- Uploaded file types: PDF, PNG, JPG only
- File size limit: define and enforce one fixed maximum
- Pagination values: validated by dependency

Business Rules

- New vendors start as `draft`
- A vendor moves to `under_review` only after at least one document is uploaded
- A vendor can move to `approved` only if every required document exists and is approved
- Rejected documents block vendor approval
- Re-uploading a document of the same type should create a new document record and mark the old one as superseded

- Reviewer name must come from the request header, not from the body
- Internal storage path must never appear in public responses

Edge Cases

- Uploading a document for a missing vendor returns `404`
- Reviewing a missing document returns `404`
- Reviewing a document twice after a final decision returns `409`
- Uploading an unsupported file type returns `415`
- Approving a vendor with missing required documents returns `409`
- Sending an empty file returns a validation or file-handling error
- Missing reviewer header returns `422` or a custom request validation error

Suggested Data Schema

Database schema is not required for this phase.

Suggested logical storage schema:

- `vendors.json`
- `documents.json`
- `review_log.json`

Core entities:

- Vendor
- VendorDocument
- DocumentReviewEvent

Expected Folder Structure

```
vendor_onboarding_api/  
  main.py  
  models.py  
  dependencies.py  
  exceptions.py  
  routers/  
    vendors.py  
    documents.py  
    reviews.py  
  data/  
    vendors.json  
    documents.json  
    review_log.json  
  uploads/  
    vendor_documents/
```

Deliverables

- Modular FastAPI project
- Working upload flow
- JSON seed data
- At least one global exception handler
- At least one custom middleware
- README with request examples

Evaluation Criteria

- Correct separation of input/output/update models
- Good use of routers and dependencies
- Safe file-upload validation and metadata handling
- Correct onboarding workflow logic
- Clean response design and consistent error handling

Bonus Challenges

- Add `response_model_exclude_none=True` where it improves public responses
- Add a cookie that stores the last selected vendor list filter for non-critical convenience behavior
- Add a middleware-generated `X-Request-ID` response header and include it in error responses
- Add a route that returns only vendors blocked by rejected documents

Project 2 - Recruitment Pipeline And Interview Coordination API

Difficulty Level

Advanced

Estimated Completion Time

12-16 hours

Project Overview

Build an API for an internal recruiting team that manages job postings, candidates, applications, interview rounds, interviewer scorecards, and resume uploads.

This project is intentionally more complex than Phase 1: it requires multiple routers, multiple models per route, response filtering, workflow rules, multipart requests, dependencies, and structured exception handling.

Problem Statement

A company needs a backend system to:

- Create job postings
- Register candidates
- Submit job applications
- Upload resumes
- Schedule interviews
- Record interviewer feedback
- Move applications through a hiring pipeline
- Search and filter applications across jobs and statuses

The API should support real recruiting workflows rather than simple CRUD operations.

Functional Requirements

- Create and manage job postings
- Create candidate records
- Upload resumes for candidates
- Create applications linking candidates to jobs
- Schedule interview rounds
- Submit interview scorecards

- Move applications through stages
- Return recruiter-friendly list endpoints with filtering, sorting, and pagination
- Return different response shapes for recruiter list views vs detailed application views

Non-Functional Requirements

- Use routers for `jobs`, `candidates`, `applications`, and `interviews`
- Use JSON files for metadata and `uploads/resumes/` for files
- Use middleware for timing and simple audit headers
- Use dependencies for reusable pagination, recruiter header parsing, and entity loading

API Requirements

- Use separate create/update/output models
- Use `response_model` to avoid leaking internal notes in list views
- Use `UploadFile` for resumes
- Use a required header such as `X-Recruiter-Name` for actions that create interviews or scorecards
- Use `Form()` for resume metadata if needed
- Use custom exception handling for invalid workflow transitions

Required API Endpoints

Method	Path	Purpose
POST	<code>/jobs</code>	Create job posting
GET	<code>/jobs</code>	List/search jobs
GET	<code>/jobs/{job_id}</code>	Get one job
PUT	<code>/jobs/{job_id}</code>	Replace job
POST	<code>/candidates</code>	Create candidate
GET	<code>/candidates</code>	List/search candidates
GET	<code>/candidates/{candidate_id}</code>	Get one candidate
POST	<code>/candidates/{candidate_id}/resume</code>	Upload resume
POST	<code>/applications</code>	Create application
GET	<code>/applications</code>	List/filter applications
GET	<code>/applications/{application_id}</code>	Get detailed application
PATCH	<code>/applications/{application_id}/stage</code>	Change stage

Method	Path	Purpose
POST	/applications/{application_id}/interviews	Schedule interview
GET	/applications/{application_id}/interviews	List interviews
POST	/interviews/{interview_id}/scorecards	Submit scorecard
GET	/dashboard/recruiting	Aggregate recruiting metrics

Request And Response Expectations

Job creation request should include:

- title
- department
- location
- employment_type
- openings
- status

Candidate creation request should include:

- full_name
- email
- phone
- years_experience
- skills

Application creation request should include:

- job_id
- candidate_id
- source
- optional note

Interview scheduling request should include:

- round_name
- scheduled_for
- interviewer_names
- mode

Scorecard request should include:

- communication_score
- technical_score
- recommendation
- summary

Validation Requirements

- Use validated enums or controlled string fields for:
 - job status
 - application stage
 - interview mode
 - recommendation
- Score fields must stay within a fixed range
- Candidate email must be unique
- Resume file type and size must be validated
- `scheduled_for` must be a valid datetime
- `interviewer_names` must be a non-empty list
- Search and pagination params should be validated by dependencies

Business Rules

- A job must be `open` before applications can be created
- A candidate cannot apply to the same job twice
- Applications start at `applied`
- Stage transitions must be controlled, for example:
 - `applied -> screening`
 - `screening -> interview`
 - `interview -> offer`
 - `offer -> hired`
 - `offer -> rejected`
- Scorecards can only be submitted for scheduled interviews
- A completed application should not accept new interviews
- Resume upload should update candidate metadata but should not expose file-system paths in responses

Edge Cases

- Applying to a missing job or candidate returns `404`
- Applying to a closed job returns `409`
- Duplicate candidate email returns `409`
- Submitting a scorecard for a missing interview returns `404`
- Scheduling an interview for a rejected application returns `409`
- Uploading an empty or invalid resume returns a file-handling error
- Missing recruiter header on recruiter actions returns a validation or custom error

Suggested Data Schema

Database schema is not required for this phase.

Suggested logical storage schema:

- `jobs.json`
- `candidates.json`
- `applications.json`
- `interviews.json`
- `scorecards.json`

Core entities:

- JobPosting
- Candidate
- Application
- Interview
- InterviewScorecard

Expected Folder Structure

```
recruitment_pipeline_api/  
  main.py  
  models.py  
  dependencies.py  
  exceptions.py  
  routers/  
    jobs.py  
    candidates.py  
    applications.py  
    interviews.py  
  data/  
    jobs.json  
    candidates.json  
    applications.json  
    interviews.json  
    scorecards.json  
  uploads/  
    resumes/
```

Deliverables

- Modular FastAPI application
- Resume upload support
- Recruiter dashboard endpoint

- Structured response models for list and detail views
- JSON seed data
- README with workflow examples

Evaluation Criteria

- Correct use of multiple models per route
- Correct response filtering for public vs internal fields
- Strong workflow validation
- Clean modular structure
- Good use of dependencies and custom exception handling

Bonus Challenges

- Add `response_model_exclude_unset=True` where partial detail responses benefit from it
- Add a dependency that validates a recruiter action header and reuses it across multiple routers
- Add a cookie storing the last selected application stage filter
- Add a list endpoint for applications missing a scorecard

Project 3 - Incident Reporting And Evidence Intake API

Difficulty Level

Very Advanced

Estimated Completion Time

14-18 hours

Project Overview

Build an internal operations API for reporting incidents, attaching evidence, assigning responders, tracking status changes, and generating incident summaries.

This project should exercise middleware, dependencies, multiple routers, file uploads, response models, custom exception handling, headers, and controlled workflow transitions.

Problem Statement

An operations team needs to track workplace or platform incidents in one backend system. They need to:

- File incidents quickly

- Attach screenshots, logs, or documents
- Assign responders
- Record timeline updates
- Filter incidents by severity and status
- Hide internal-only notes from standard incident responses

Functional Requirements

- Create incidents
- List incidents with searching, filtering, sorting, and pagination
- View incident detail
- Update editable incident fields
- Upload evidence files
- Add timeline events
- Assign or reassign responder names
- Move incidents through a controlled status workflow
- Return summary metrics for open vs resolved incidents

Non-Functional Requirements

- Use routers such as `incidents`, `evidence`, and `timeline`
- Use consistent exception handling
- Add middleware for request ID and response time
- Keep file metadata separate from file paths in outward responses
- Use dependencies for incident lookup and common filters

API Requirements

- Accept evidence using multipart form data
- Read an operator name from a header such as `X-Operator-Name`
- Use distinct output models for list view and detail view
- Use validators for severity, status, and timeline event rules
- Use custom exception classes for invalid status transitions and evidence problems

Required API Endpoints

Method	Path	Purpose
POST	<code>/incidents</code>	Create incident
GET	<code>/incidents</code>	List/search/filter incidents

Method	Path	Purpose
GET	/incidents/{incident_id}	Get incident detail
PUT	/incidents/{incident_id}	Replace editable incident fields
PATCH	/incidents/{incident_id}/status	Change incident status
POST	/incidents/{incident_id}/assign	Assign responder
POST	/incidents/{incident_id}/evidence	Upload evidence
GET	/incidents/{incident_id}/evidence	List evidence
POST	/incidents/{incident_id}/timeline	Add timeline event
GET	/incidents/{incident_id}/timeline	List timeline
GET	/dashboard/incidents	Return aggregate metrics

Request And Response Expectations

Incident creation request should include:

- title
- description
- severity
- service_area
- reported_by

Status update request should include:

- status
- optional note

Evidence upload request should include:

- evidence_type
- optional caption
- file

Timeline event request should include:

- event_type
- message
- visible_to_reporter

Validation Requirements

- severity: controlled values such as low, medium, high, critical
- status: controlled values such as new, triaged, in_progress, resolved, closed

- `service_area`: controlled string list
- `reported_by`: 2-100 characters
- Timeline message length constraints
- Evidence file type restrictions
- Evidence file size restrictions
- Pagination/sort parameters validated through dependencies

Business Rules

- Incidents start in `new`
- Only allowed status transitions should pass
- Resolved incidents require at least one timeline event describing the resolution
- Closed incidents cannot accept new evidence
- List responses should hide internal-only timeline notes if the endpoint is not a detail view
- Responder assignment should create a timeline event automatically
- Evidence responses must not expose internal storage paths
- Operator name must come from a request header for actions that change state

Edge Cases

- Uploading evidence to a missing incident returns `404`
- Uploading evidence to a closed incident returns `409`
- Invalid status transition returns `409`
- Missing operator header on state-changing endpoints returns a custom validation error
- Empty file uploads return a file validation error
- Timeline creation for a missing incident returns `404`
- Requests with unsupported content type should return `415` where applicable

Suggested Data Schema

Database schema is not required for this phase.

Suggested logical storage schema:

- `incidents.json`
- `evidence.json`
- `timeline.json`

Core entities:

- Incident
- EvidenceItem
- IncidentTimelineEvent

Expected Folder Structure

```
incident_reporting_api/  
  main.py  
  models.py  
  dependencies.py  
  exceptions.py  
  middleware.py  
  routers/  
    incidents.py  
    evidence.py  
    timeline.py  
  data/  
    incidents.json  
    evidence.json  
    timeline.json  
  uploads/  
    evidence/
```

Deliverables

- Modular FastAPI application
- Valid evidence upload flow
- Global exception handling
- Middleware-backed request tracing
- JSON seed data
- README covering incident lifecycle

Evaluation Criteria

- Correct workflow enforcement
- Clear separation of public and internal response fields
- Appropriate use of middleware vs dependencies
- Strong file and header validation
- Maintainable router and model structure

Bonus Challenges

- Add a middleware-generated `X-Incident-Trace` header
- Add a route returning only incidents waiting for triage
- Add a response model for dashboard widgets separate from incident detail
- Add a cookie that stores the caller's last selected severity filter

Project 4 - Editorial Workflow And Asset Approval API

Difficulty Level

Very Advanced

Estimated Completion Time

16-22 hours

Project Overview

Build a backend API for an editorial team that manages content submissions, review rounds, editorial comments, asset uploads, publishing decisions, and public-safe response views.

This project should feel like a mini production system: multiple routers, layered validation, custom exception handling, response models, file uploads, headers, cookies, middleware, and several dependent workflows.

Problem Statement

An editorial organization needs a backend platform to:

- Receive article submissions
- Upload cover assets and supporting files
- Assign editors
- Record editorial comments
- Track review rounds
- Approve, reject, or request revision
- Publish content to a public-safe endpoint view

The same content must be represented differently for submitters, editors, and public API consumers without requiring real authentication.

Functional Requirements

- Create submissions
- List and search submissions
- View detailed submission metadata
- Upload cover assets or attachments
- Add editorial comments
- Assign editors
- Move content through workflow states

- Return a public-facing published-content endpoint
- Support draft preview behavior using a non-security cookie

Non-Functional Requirements

- Use routers such as `submissions`, `assets`, `comments`, and `public`
- Use middleware for request metadata and processing time
- Use response models to produce separate public and editorial views
- Use exception handlers for business-rule and validation consistency
- Use local JSON files and upload folders only

API Requirements

- Read editor identity from a header such as `X-Editor-Name`
- Use multipart upload for assets
- Use separate models for:
 - submission create
 - submission update
 - editorial detail
 - public published view
- Use dependencies for submission loading, common filters, and editor header extraction
- Use cookies for draft preview preference only, not as authentication

Required API Endpoints

Method	Path	Purpose
POST	<code>/submissions</code>	Create submission
GET	<code>/submissions</code>	List/search/filter submissions
GET	<code>/submissions/{submission_id}</code>	Get editorial detail
PUT	<code>/submissions/{submission_id}</code>	Replace editable submission fields
POST	<code>/submissions/{submission_id}/assign-editor</code>	Assign editor
PATCH	<code>/submissions/{submission_id}/status</code>	Change workflow status
POST	<code>/submissions/{submission_id}/comments</code>	Add editorial comment
GET	<code>/submissions/{submission_id}/comments</code>	List editorial comments
POST	<code>/submissions/{submission_id}/assets</code>	Upload asset

Method	Path	Purpose
GET	/submissions/{submission_id}/assets	List assets
GET	/public/articles	List published articles
GET	/public/articles/{submission_id}	Get public-safe published article
POST	/preferences/preview-mode	Set preview cookie

Request And Response Expectations

Submission create request should include:

- title
- summary
- body
- author_name
- tags

Comment request should include:

- comment_type
- message
- visible_to_author

Asset upload request should include:

- asset_type
- optional caption
- file

Status change request should include:

- target status
- optional note

Public article responses should expose only published-safe fields and must exclude editorial notes, asset file paths, and internal review metadata.

Validation Requirements

- Title, summary, and body length constraints
- Tag list length and item validation
- comment_type: controlled values
- status: controlled workflow values such as draft, review, revision_requested, approved, published, rejected
- Asset file types and size limits
- Editor header validation on editor-only actions

- Search, filter, sort, and pagination dependency validation

Business Rules

- New submissions start as `draft`
- Only assigned editors can move a submission from `review` to another editorial state, but this should be modeled as validated request context, not real auth
- A submission cannot be published unless it has at least one approved asset and at least one editorial comment
- Rejected submissions cannot be published
- Public routes must return only published submissions
- Draft preview cookie may alter non-critical response behavior, such as whether unpublished but approved summaries appear in a preview list
- Internal comment visibility must be respected in outward responses

Edge Cases

- Publishing a non-approved submission returns `409`
- Uploading an asset to a missing submission returns `404`
- Adding an editorial comment without required editor header returns a custom validation error
- Returning a public view for an unpublished submission returns `404`
- Unsupported asset type returns `415`
- Empty asset upload returns a file validation error
- Invalid status transitions return `409`

Suggested Data Schema

Database schema is not required for this phase.

Suggested logical storage schema:

- `submissions.json`
- `comments.json`
- `assets.json`
- `editor_assignments.json`

Core entities:

- Submission
- EditorialComment
- SubmissionAsset
- EditorAssignmentHistory

Expected Folder Structure

```
editorial_workflow_api/  
  main.py  
  models.py  
  dependencies.py  
  exceptions.py  
  middleware.py  
  routers/  
    submissions.py  
    comments.py  
    assets.py  
    public.py  
    preferences.py  
  data/  
    submissions.json  
    comments.json  
    assets.json  
    editor_assignments.json  
  uploads/  
    editorial_assets/
```

Deliverables

- Modular FastAPI application
- Public-safe published-content API
- Asset upload support
- Editorial workflow endpoints
- JSON seed data
- README with submission-to-publication flow

Evaluation Criteria

- Strong separation between editorial and public response models
- Good use of dependencies, middleware, and routers
- Correct workflow enforcement
- Clean handling of headers, cookies, and file uploads
- Consistent exception handling and response structure

Bonus Challenges

- Add a middleware response header such as `X-Editorial-Request-ID`
- Add a route returning submissions waiting on revision

- Add separate response models for summary cards and detailed editorial views
- Add a router-level dependency for editor actions

Final Submission Checklist

For each project, submit:

- Full project folder
- `main.py`
- router modules
- model/schema modules
- dependency module
- exception-handling module if used
- JSON seed data files
- upload folder structure
- README with example requests and workflow explanation

Before submitting, verify:

- The API starts without errors
- `/docs` shows all routes
- Request validation works
- Response models hide internal-only fields
- Middleware behavior is visible in responses where expected
- File uploads validate type and size correctly
- Custom exceptions produce the expected error shape
- Dependencies remove duplication instead of adding confusion
- No Phase 3+ topic was required

Difficulty Progression

Project	Difficulty	Main Focus
Vendor Onboarding And Compliance Review API	Hard	Routers, file uploads, response models, workflow rules
Recruitment Pipeline And Interview Coordination API	Advanced	Multiple models, workflows, recruiter context, resume uploads
Incident Reporting And Evidence Intake API	Very Advanced	Middleware, evidence handling, public/internal views, timeline workflows
Editorial Workflow And Asset Approval API	Very Advanced	Multi-view responses, asset uploads, editorial workflow, cookies and headers

Complete the projects in order. The later projects assume you can already combine Phase 1 fundamentals with Phase 2 architecture features naturally.